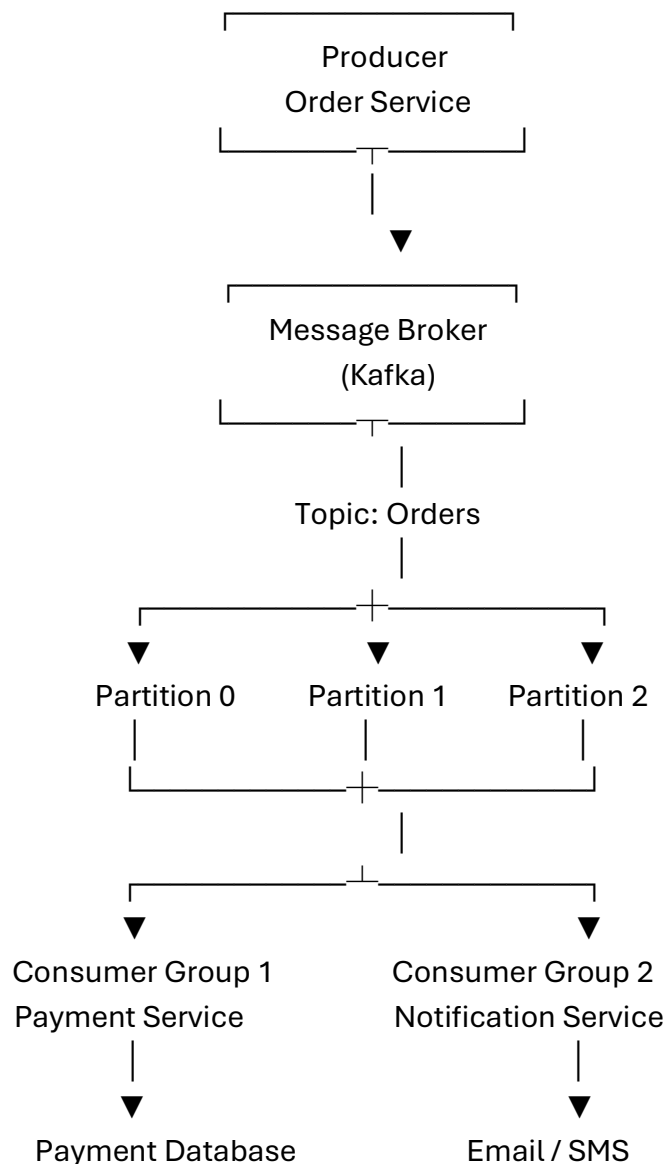


EPIC 07 Task 04 - Messaging & Event-Driven Architecture

Objective

Design an event-driven system where services communicate through a **message broker** such as Apache Kafka or RabbitMQ.

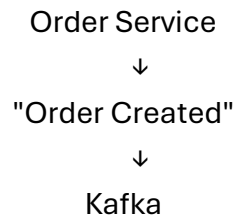
Architecture



1. Producers

A **producer** creates and publishes events/messages.

Example:



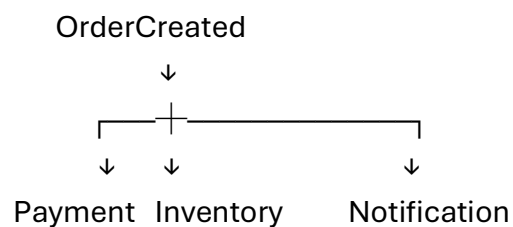
When a customer places an order, the Order Service publishes an OrderCreated event.

2. Consumers

A **consumer** receives and processes messages from a topic.

Examples:

- Payment Service → processes payment.
- Inventory Service → updates stock.
- Notification Service → sends confirmation.



3. Topics

A **topic** is a logical category where events are stored.

Example:

orders
payments
notifications
inventory

For example:

orders topic
|
├— OrderCreated
├— OrderUpdated
└— OrderCancelled

4. Partitions

A topic can be divided into multiple **partitions** to improve scalability and parallel processing.

Orders Topic
├— Partition 0
├— Partition 1
└— Partition 2

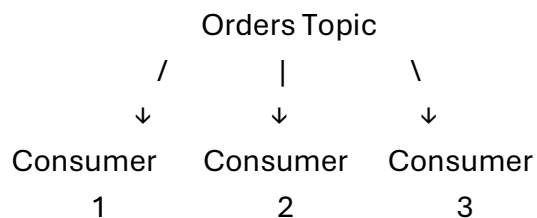
Different consumers can process different partitions simultaneously.

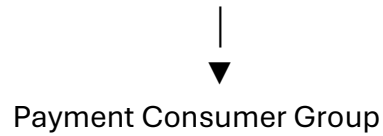
Benefit: Higher throughput and better scalability.

5. Consumer Groups

A **consumer group** is a set of consumers working together to process messages.

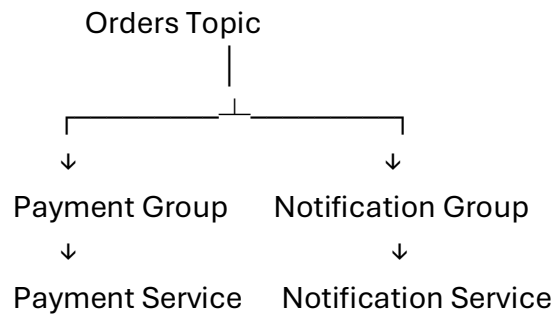
Example:





Within one consumer group, a partition is processed by only one consumer at a time.

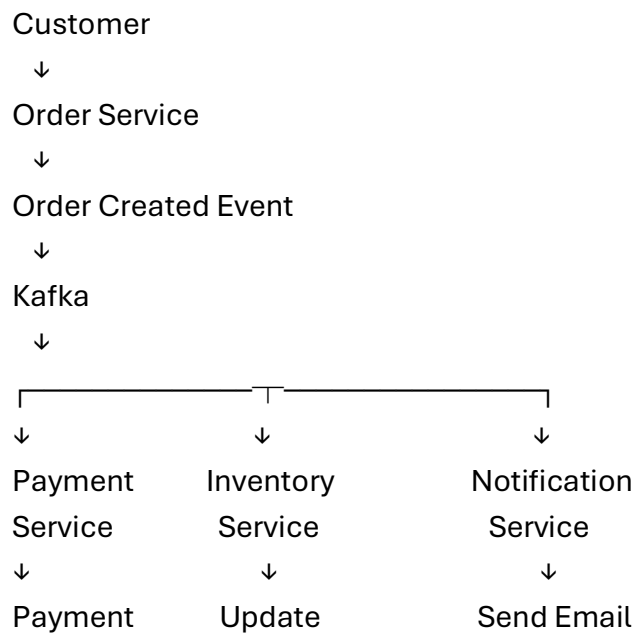
Multiple consumer groups can independently consume the same topic:



6. Event Processing

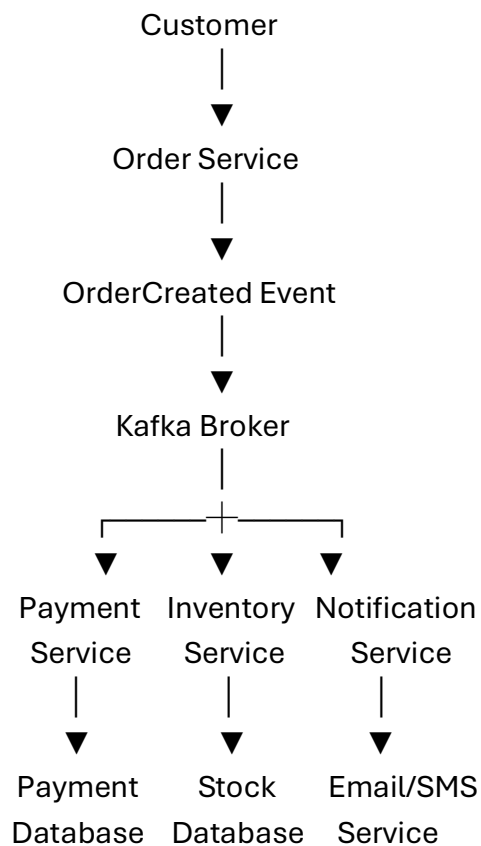
Event processing means consuming an event and performing an action.

Example: Order Creation



This allows services to communicate **asynchronously** without directly depending on each other.

7. Complete Event-Driven Flow



Advantages

- **lose coupling** between services.
- **Asynchronous communication.**
- **High scalability.**
- **Better fault tolerance.**
- **Parallel event processing.**
- Services can process events independently.

Key Terms

Term	Meaning
Producer	Creates and publishes events
Consumer	Reads and processes events
Topic	Category/storage for events
Partition	Subdivision of a topic for parallel processing
Consumer Group	Consumers working together
Event	A record of something that happened

Task 04 is completed with a Kafka-based event-driven architecture using producers, consumers, topics, partitions, consumer groups, and event processing.